La pila (Stack): SplStack

Caso práctico

Carlos está programando el editor de descripciones de productos de la tienda de **BK Programación** y quiere añadir el botón **Deshacer**. Lo primero que se le ocurre es guardar todos los cambios en un array y buscar "el último"... **María** le dice que eso tiene un nombre: una **pila**, y que PHP ya trae una implementación en la SPL, `SplStack`.

1.- Qué es una pila

Una **pila** (*stack*) es un tipo de dato abstracto (ADT) en el que los elementos se añaden y se quitan **siempre por el mismo extremo**, la **cima**. El último elemento que entra es el primero que sale: **LIFO** (*Last In, First Out*). Es como una pila de platos: se pone uno encima y se coge el de arriba.

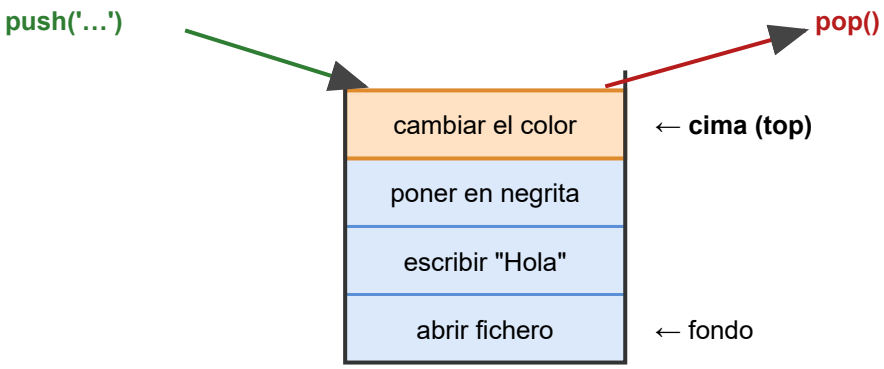


Figura 1. Una pila: push mete en la cima, pop saca de la cima

Las operaciones del ADT son:

Operación	Qué hace	En SplStack	Coste
push	Meter un elemento en la cima	`push($x)` (o `$pila[] = $x`)	O(1)
pop	Sacar el elemento de la cima y devolverlo	`pop()`	O(1)
top / *peek*	Ver el elemento de la cima sin sacarlo	`top()`	O(1)
isEmpty	¿Está vacía?	`isEmpty()`	O(1)
size	Número de elementos	`count($pila)`	O(1)

O(1) significa que la operación tarda lo mismo con 10 elementos que con un millón (apartado 1 de la introducción).

2.- La clase SplStack

`SplStack` hereda de `SplDoublyLinkedList` (la lista doblemente enlazada) fijando su modo de recorrido en LIFO, e implementa `Iterator`, `Countable`, `ArrayAccess` y `Serializable`.

```
<?php
$acciones = new SplStack();
$acciones->push('escribir "Hola"');
$acciones->push('poner en negrita');
$acciones->push('cambiar el color');

echo $acciones->top();          // "cambiar el color" (solo mirar)
echo $acciones->pop();          // "cambiar el color" (deshacer)
echo $acciones->pop();          // "poner en negrita"
echo count($acciones);          // 1
```

Debes conocer

Con una pila **vacía**, `pop()` y `top()` lanzan `RuntimeException` ("Can't pop from an empty datastructure"). Comprueba antes `isEmpty()` o captura la excepción. Un array, en cambio, devuelve `null` en silencio con `array_pop([])`, y el error pasa desapercibido.

3.- Recorrer una pila

- `foreach` recorre la pila de la **cima al fondo** y **no** saca los elementos (modo `IT_MODE_KEEP`). Las **claves** que da son las posiciones contadas desde el fondo (con 4 elementos: 3, 2, 1, 0).
- Con `$pila[0]` se accede a la **cima**, `$pila[1]` al de debajo... ¡Cuidado: es al revés que las claves del `foreach`!
- `SplStack` **no tiene** `toArray()`: para obtener un array se usa `iterator_to_array($pila, false)` (de la cima al fondo).
- Con `setIteratorMode(SplDoublyLinkedList::IT_MODE_LIFO | SplDoublyLinkedList::IT_MODE_DELETE)` el `foreach` va **sacando** los elementos. El sentido (LIFO) de una `SplStack` no se puede cambiar: intentar ponerla en FIFO lanza `RuntimeException`.

4.- ¿SplStack o un array?

Un array de PHP también sirve como pila: `array_push($a, $x)` (o `$a[] = $x`), `array_pop($a)` y `end($a)` para ver la cima, todo O(1). `SplStack` aporta:

- un **objeto con nombre** que deja claro qué es y qué operaciones admite (`top`, `isEmpty`), y que se puede usar como **tipo** (`function deshacer(SplStack $acciones)`),
- **excepciones** si se saca de una pila vacía,
- y es un objeto: se pasa a las funciones sin copiarse (el array se copia).

Para pilas pequeñas y código sencillo, un array está bien; cuando la pila es parte del diseño (un editor, un intérprete), `SplStack` comunica mejor la intención.

5.- Aplicaciones típicas

Problema	Por qué una pila
Deshacer / rehacer	La última acción es la primera que se deshace (dos pilas: deshacer y rehacer)
Botón "Atrás" del navegador	Se vuelve a la última página visitada
Paréntesis o etiquetas HTML equilibrados	Cada cierre debe corresponder a la última apertura pendiente
Calculadoras (notación polaca inversa)	Los operandos se apilan; el operador usa los dos últimos
Recorridos en profundidad (carpetas, laberintos, grafos)	Se avanza por un camino hasta el final y se retrocede al último punto pendiente
Llamadas a funciones (la *pila de llamadas*)	Cada llamada se apila; al terminar, se vuelve a la anterior

```php
<?php
// Calculadora en notación polaca inversa: "3 4 + 2 *" = (3 + 4) * 2
function evaluarRpn(string $expresion): float
{
    $pila = new SplStack();
    foreach (preg_split('/\s+/', trim($expresion)) as $token) {
        if (is_numeric($token)) {
            $pila->push((float) $token);
            continue;
        }
        $b = $pila->pop();                    // el de arriba es el SEGUNDO operando
        $a = $pila->pop();
        $pila->push(match ($token) { '+' => $a + $b, '-' => $a - $b, '*' => $a * $b, '/' => $a
/ $b });
    }
    return $pila->pop();
}
echo evaluarRpn('3 4 + 2 *');                 // 14
```

6.- Ejercicio resuelto: deshacer y rehacer

Ejercicio resuelto

Crea una clase `Editor` con `escribir()`, `deshacer()` y `rehacer()`, como el Ctrl+Z y el Ctrl+Y de cualquier editor.

```php
<?php
final class Editor
{
```

```
    private string $texto = '';
    private SplStack $deshacer;
    private SplStack $rehacer;

    public function __construct()
    {
        $this->deshacer = new SplStack();
        $this->rehacer = new SplStack();
    }

    public function escribir(string $trozo): void
    {
        $this->deshacer->push($this->texto);     // guardamos el estado ANTERIOR
        $this->texto .= $trozo;
        $this->rehacer = new SplStack();         // un cambio nuevo invalida el "rehacer"
    }

    public function deshacer(): void
    {
        if (!$this->deshacer->isEmpty()) {
            $this->rehacer->push($this->texto);
            $this->texto = $this->deshacer->pop();
        }
    }

    public function rehacer(): void
    {
        if (!$this->rehacer->isEmpty()) {
            $this->deshacer->push($this->texto);
            $this->texto = $this->rehacer->pop();
        }
    }

    public function texto(): string
    {
        return $this->texto;
    }
}
$editor = new Editor();
$editor->escribir('Hola');
$editor->escribir(', mundo');
$editor->escribir('!');
$editor->deshacer();
$editor->deshacer();
echo $editor->texto();          // "Hola"
$editor->rehacer();
echo $editor->texto();          // "Hola, mundo"
```

Observa que las dos pilas son **privadas**: desde fuera solo se puede escribir, deshacer y rehacer, y nadie puede dejar el editor en un estado incoherente.

Autoevaluación

- Si se hace `push('a')`, `push('b')`, `push('c')` y después `pop()`, ¿qué se obtiene? **"c": el último en entrar.**
- ¿Qué diferencia hay entre `top()` y `pop()`? **`top()` solo mira la cima; `pop()` la saca.**
- ¿Qué ocurre al hacer `pop()` en una `SplStack` vacía? **Lanza `RuntimeException`.**
- En una `SplStack`, ¿`$pila[0]` es la cima o el fondo? **La cima.**
- ¿Cómo obtienes un array con el contenido de una `SplStack`? **`iterator_to_array($pila, false)` (no existe `toArray()`).**

7.- Ejercicios

Fichero	Contenido
`Ejercicios Stack/e01-SplStack.php`	La API: `push`, `top`, `pop`, pila vacía, `foreach` y posiciones, `IT_MODE_DELETE`, comparación con un array
`Ejercicios Stack/e02-SplStack.php`	Aplicaciones: deshacer y rehacer, HTML bien anidado, notación polaca inversa, recorrido en profundidad sin recursividad
`Ejercicios Stack/problemas/`	`e01` y `e02`: problemas (`_problema.php`) y soluciones (`_problema_solucion.php`)